

ElGamals

AI Usage statement: No AI was used in the creation of this program.

1. ElGamal Encryption - Encrypt the following messages using ElGamal encryption (Z_{409} and $g = 19$):

Perform both encryption and decryption. Show every intermediate step of the ElGamal Scheme.

1. Private key $x = 301$, ephemeral key $k = 23$, message $m = 59$.

1. `go run .\main.go 301 23 59`

```
Test values: q=409, g=19, x=301, k=23, m=59
h (g^x mod q): 398
g^k used : 351
g^ak used : 258
Encrypted message: 15222
Ephemeral public key p: 351
Decrypted message: 59
```

2. Private key $x = 301$, ephemeral key $k = 135$, message $m = 59$.

1. `go run .\main.go 301 135 59`

```
Test values: q=409, g=19, x=301, k=135, m=59
h (g^x mod q): 398
g^k used : 366
g^ak used : 223
Encrypted message: 13157
Ephemeral public key p: 366
Decrypted message: 59
```

3. Considering the examples from problem 1, we see that the ElGamal encryption scheme is non-deterministic: A given message m has many valid encryptions

1. Why is the ElGamal encryption scheme non-deterministic?

1. ElGamal encryption chooses a new random ephemeral key k for every encryption. Different random k produce different pairs (g^k, h^k) even for the same m , so the ciphertexts differ.

2. How many valid ciphertexts are there for each message m (general expression)? How many are there for the system in problem 1 (numerical answer)?

1. General expression: the number of distinct ciphertexts for a fixed message m equals the number of allowed choices for the ephemeral key k . If K is the set of possible k values, then number of ciphertexts = $|K|$.

2. Numerical answer for problem 1 (with $p = 409$): using the common choice $k \in \{1, \dots, p-2\}$, the number of valid ciphertexts for each m is $409 - 2 = 407$ (if you instead allow $k \in \{1, \dots, p-1\}$ except 0, you'd get 408).

3. Consider the case that for two messages $m_1 \neq m_2$ the same ephemeral key $k_1 = k_2$ has been chosen for the ElGamal encryption. This kind of behavior occurs if no or a bad

cryptographic PRNG is being used. Show how the message m_2 can be recovered from a known message ciphertext pair m_1, c_1 if the same k is used.

1. Let the two ciphertexts (for messages m_1 and m_2) be:
 1. $C_1 = (p = g^k \bmod q, c_{2_1} = m_1 * h^k \bmod q)$
 2. $C_2 = (p = g^k \bmod q, c_{2_2} = m_2 * h^k \bmod q)$
2. If an attacker knows the plaintext m_1 and the ciphertext C_1 , they can compute h^k :
 1. $h^k \equiv c_{2_1} * m_1^{-1} \pmod{q}$ (because $c_{2_1} = m_1 * h^k$)
3. Then recover m_2 from C_2 :
 1. $m_2 \equiv c_{2_2} * (h^k)^{-1} \pmod{q}$
 2. Substitute h^k : $m_2 \equiv c_{2_2} * (c_{2_1} * m_1^{-1})^{-1} \equiv c_{2_2} * c_{2_1}^{-1} * m_1 \pmod{q}$
4. So with one known plaintext/ciphertext pair and reuse of k , an attacker obtains m_2 by computing
 1. $m_2 = m_1 \cdot c_{2_2} \cdot \text{inv}(c_{2_1}) \pmod{q}$

4. Implement the square and multiply algorithm using a computer language of your choice

```
// modExp computes (base^exp) % mod
import "math/big"
func modExp(base, exp, mod *big.Int) *big.Int {
    // result := new(big.Int).Exp(base, exp, mod)

    // Implement modular exponentiation using binary exponentiation (square-and-
multiply)
    // Work on copies so we don't mutate caller's values
    e := new(big.Int).Set(exp)
    b := new(big.Int).Mod(new(big.Int).Set(base), mod)
    result := big.NewInt(1)

    zero := big.NewInt(0)

    for e.Cmp(zero) > 0 {
        // if least significant bit is 1, multiply result by current base
        if e.Bit(0) == 1 {
            result.Mul(result, b)
            result.Mod(result, mod)
        }
        // square base: b = b*b mod mod
        b.Mul(b, b)
        b.Mod(b, mod)
        // shift exponent right by 1
        e.Rsh(e, 1)
    }
    return result
}
```

1. (a) $a = 235973, b = 456872884723247, N = 583884$
 1. 193541
2. (b) $a = 984327455683, b = 1253489582, N = 994348472542$

1. 889817920395